

Guía práctica actualizar (UPDATE) y eliminar (DELETE) registros en MySQL.

1. Preparación del entorno

Crearemos una base de datos con una tabla de prueba:

```
CREATE DATABASE tienda3;
USE tienda3;

CREATE TABLE productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50),
  precio DECIMAL(10,2),
  stock INT
);

INSERT INTO productos (nombre, precio, stock) VALUES
('Manzana', 50.00, 100),
('Banana', 30.00, 50),
('Naranja', 40.00, 80);
```

2. Actualizar registros con UPDATE

El comando UPDATE se utiliza para modificar uno o varios campos de registros existentes.

Sintaxis básica:

```
UPDATE nombre_tabla
SET columna1 = valor1, columna2 = valor2
WHERE condición;
```

 **Importante:** Siempre usa WHERE para evitar modificar TODOS los registros.

Ejemplo 1: Actualizar el precio de la "Manzana"

```
UPDATE productos
SET precio = 55.00
WHERE nombre = 'Manzana';
```

✓ Ejemplo 2: Aumentar el stock a 200 unidades para todos los productos con stock menor a 60

```
UPDATE productos  
SET stock = 200  
WHERE stock < 60;
```



3. Eliminar registros con DELETE

El comando DELETE elimina registros de una tabla según una condición.



Sintaxis básica:

```
DELETE FROM nombre_tabla  
WHERE condición;
```



¡Cuidado! Sin WHERE, eliminarás TODOS los registros.



Ejemplo 1: Eliminar el producto "Naranja"

```
DELETE FROM productos  
WHERE nombre = 'Naranja';
```



Ejemplo 2: Eliminar productos con precio menor a 35

```
DELETE FROM productos  
WHERE precio < 35;
```

Consejos prácticos

- Siempre verifica con un **SELECT** antes de ejecutar **UPDATE** o **DELETE**:
- `SELECT * FROM productos WHERE precio < 35;`
- **Haz siempre backups de tu base de datos** cuando trabajes con datos importantes.

Ejercicio de práctica

1. Inserta un nuevo producto llamado “Pera” con precio 45.00 y stock 60.
2. Actualiza el stock de “Banana” a 80.
3. Elimina todos los productos cuyo nombre empiece con "M".

Guía con ejemplos de UPDATE o DELETE usando JOIN, subconsultas y transacciones

Tablas de ejemplo

```
DROP DATABASE IF EXISTS tienda4;
CREATE DATABASE tienda4;

USE tienda4;

-- Tabla de categorías con InnoDB
CREATE TABLE categorias (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50)
) ENGINE=InnoDB;

-- Tabla de productos con InnoDB y FK
CREATE TABLE productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50),
  precio DECIMAL(10,2),
  stock INT,
  categoria_id INT,
  FOREIGN KEY (categoria_id) REFERENCES categorias(id)
) ENGINE=InnoDB;

-- Insertar datos
INSERT INTO categorias (nombre) VALUES
('Frutas'),
('Verduras'),
('Bebidas');

INSERT INTO productos (nombre, precio, stock, categoria_id) VALUES
('Manzana', 50.00, 100, 1),
('Banana', 30.00, 50, 1),
('Zanahoria', 20.00, 80, 2),
('Agua Mineral', 60.00, 120, 3);
```

1. UPDATE con JOIN

Podés actualizar registros basados en información de otra tabla usando JOIN.

✓ **Ejemplo: Aumentar el precio de productos de la categoría "Frutas" en 10%**

```
UPDATE productos p
JOIN categorias c ON p.categoria_id = c.id
SET p.precio = p.precio * 1.1
WHERE c.nombre = 'Frutas';
```

2. UPDATE con Subconsulta

Las subconsultas permiten usar valores calculados o datos devueltos desde otra consulta.

✓ **Ejemplo: Actualizar el stock de un producto al promedio de stock de todos los productos**

```
UPDATE productos
SET stock = (SELECT AVG(stock) FROM productos)
WHERE nombre = 'Zanahoria';
```

Aquí puede ocurrir un error, este error ocurre porque MySQL no permite que uses la misma tabla (**productos**) en un **UPDATE** y al mismo tiempo en un **subquery** (subconsulta) **del FROM o el SELECT**. Es una restricción para evitar resultados impredecibles.

```
UPDATE productos
SET stock = (
  SELECT avg_stock
  FROM (
    SELECT AVG(stock) AS avg_stock
    FROM productos
  ) AS temp
)
WHERE nombre = 'Zanahoria';
```



3. DELETE con JOIN

Podés eliminar datos asociados a otra tabla sin necesidad de hacer múltiples consultas.



Ejemplo: Eliminar todos los productos de la categoría "Verduras"

```
DELETE p
FROM productos p
JOIN categorias c ON p.categoria_id = c.id
WHERE c.nombre = 'Verduras';
```



4. DELETE con Subconsulta

También podés borrar registros basados en un criterio de otra tabla.



Ejemplo: Eliminar productos con precio inferior al precio promedio

```
DELETE FROM productos
WHERE precio < (
    SELECT avg_precio
    FROM (
        SELECT AVG(precio) AS avg_precio
        FROM productos
    ) AS temp
);
```



5. Uso de Transacciones (BEGIN / COMMIT / ROLLBACK)

Las transacciones te permiten ejecutar un conjunto de cambios y deshacerlos si hay algún error.

```
START TRANSACTION;

UPDATE productos
SET precio = precio * 0.5
WHERE stock > 100;

DELETE FROM productos
WHERE stock <= 50;

-- Si todo fue correcto:
COMMIT;

-- Si algo salió mal, usar en su lugar:
-- ROLLBACK;
```



¿Qué hace cada parte?

1. **START TRANSACTION;**
Inicia una transacción. Desde aquí, los cambios no se guardan de forma permanente hasta que uses COMMIT.
2. **UPDATE y DELETE**
Operaciones que se incluyen dentro de la transacción. Podrías tener más operaciones aquí si fuera necesario.
3. **COMMIT;**
Confirma la transacción. Todos los cambios se guardan definitivamente en la base de datos.
4. **ROLLBACK;**
Sirve para deshacer todos los cambios desde START TRANSACTION, en caso de que notes un error o quieras cancelar la operación.

Detalle para mayor control

Antes del COMMIT, podés hacer una **verificación** con un **SELECT** para evaluar los resultados **antes** de confirmarlos:

```
START TRANSACTION;

UPDATE productos
SET precio = precio * 0.5
WHERE stock > 100;

DELETE FROM productos
WHERE stock <= 50;

-- Revisá los cambios previos
SELECT * FROM productos;

-- Si estás conforme:
COMMIT;

-- Si algo no estuvo bien: ROLLBACK;
```

Recordatorio

- El motor de la tabla debe ser **InnoDB** para soportar transacciones.
- SHOW TABLE STATUS WHERE Name = 'productos';
- En líneas comentadas (--), podés activar el ROLLBACK o el COMMIT según la situación.

Buenas prácticas

- Trabajá con **copias de seguridad** antes de aplicar cambios importantes.
- Siempre hacé un **SELECT previo** para confirmar qué registros se verán afectados.
- Cuanto más complejas sean las sentencias, más recomendable es usar **transacciones**.



Ejercicio propuesto

1. Aumentar en un 15% el precio de todos los productos que tengan más de 80 de stock y pertenezcan a la categoría “Bebidas”.
2. Eliminar todos los productos cuyo precio sea el máximo de su categoría.
3. Usar una transacción para hacer un UPDATE y un DELETE y probar ROLLBACK si algo no parece correcto.